

Arquitectura de Software de Spotify

Investigación de Arquitectura y Rol del Arquitecto de Software

Daniel Alberto Serrano Mora
Andrés Jose Westra Ureña
Carlos Andres Fuentes Santos
Erick Andre Campos Sibaja
Michael Ramírez González

■ Agenda

- Sección 1: Investigación de arquitectura de Spotify
- Sección 2: Rol del arquitecto de software actual
- Sección 3: El rol del arquitecto en el futuro

■ Objetivo del Trabajo

- Investigar la **arquitectura de software de Spotify** como plataforma digital de escala global
- Analizar el **rol actual del arquitecto de software** en el mercado laboral
- Proponer la **evolución del rol en los próximos 10 años** considerando inteligencia artificial y computación cuántica

■ Introducción

El presente trabajo analiza la **arquitectura de software de Spotify** desde tres perspectivas:

- **Sección 1:** Arquitectura actual – microservicios, tecnologías, datos, seguridad
- **Sección 2:** Rol del arquitecto de software en el mercado laboral actual
- **Sección 3:** Evolución del rol en la próxima década (IA, computación cuántica)

■ Spotify en Cifras

- **600M+** usuarios activos mensuales
- **100M+** pistas en el catálogo
- **~1.500** microservicios
- **250+** squads autónomos
- **1 billón+** de eventos diarios procesados
- Infraestructura sobre **Google Cloud Platform**

■ Sección 1: Arquitectura de Spotify

▤ Tipo de Arquitectura

- **Microservicios** (~1.500 servicios independientes)
- **Event-Driven** con Apache Kafka como backbone
- **Comunicación dual**: REST/gRPC (síncrono) + Kafka/PubSub (asíncrono)
- **Backend for Frontend**: APIs por tipo de cliente
- **API Gateway**: Apollo GraphQL
- **Strangler Fig**: migración progresiva desde el monolito

■ Sección 1: Arquitectura de Spotify

▤ Patrones Arquitectónicos

- **Microservicios**: despliegue y BD independientes por servicio
- **Event-Driven**: Kafka procesa billones de eventos/día
- **CQRS**: separación lectura/escritura en catálogo y playlists
- **Saga / Coreografía**: transacciones distribuidas sin orquestador
- **CDN / Edge Delivery**: distribución global multi-proveedor

■ Sección 1: Arquitectura de Spotify

▤ Tecnologías Principales – Lenguajes

- **Java** + Spring Boot: backend predominante
- **Python**: ML, ciencia de datos, automatización
- **C++**: motor de streaming, reproductores nativos
- **Scala** (Scio): pipelines de datos sobre Apache Beam
- **TypeScript** / **Node.js**: frontend web y servicios ligeros

■ Sección 1: Arquitectura de Spotify

■ Tecnologías – Infraestructura

- **Google Cloud Platform:** migración total desde datacenters propios (2016-2018)
- **Kubernetes** (GKE): orquestación de contenedores
- **Docker:** empaquetado estándar
- **CDN multi-proveedor:** puntos de presencia propios
- **CI/CD:** Jenkins + Spinnaker

■ Sección 1: Arquitectura de Spotify

▒ Tecnologías – Almacenamiento y Mensajería

- **Apache Cassandra** (~3.000 nodos): catálogo, metadatos
- **PostgreSQL**: usuarios, cuentas, facturación
- **BigTable / BigQuery**: métricas y analítica
- **Redis / Memcached**: caché distribuida
- **Elasticsearch**: búsqueda full-text
- **Apache Kafka**: >1 billón de eventos/día

■ Sección 1: Arquitectura de Spotify

▤ Componentes de la Solución



■ Sección 1: Arquitectura de Spotify

▤ Catálogo y Recomendaciones

Catálogo Musical

- Metadata Service sobre Cassandra
- Audio Features Pipeline (ML)
- Search Service con Elasticsearch
- Cover Art Service

Motor de Recomendaciones

- BaRT: Bandits multi-armados
- Discover Weekly / Release Radar
- Daily Mix
- Taste Profile: vectores densos
- DJ AI (2023): IA generativa

■ Sección 1: Arquitectura de Spotify

▒ Streaming y Publicidad

Streaming de Audio

- Códecs: OGG Vorbis, AAC
- VBR adaptativo a red
- Spotify Connect
- Offline con cifrado local

Publicidad (Free Tier)

- Ad Insertion dinámico
- Segmentación de audiencias
- Subasta en tiempo real
- Analytics de atribución

■ Sección 1: Arquitectura de Spotify

■ Social, Auth y Plataforma de Datos

Social y Playlists

- Playlist Service + CRDTs
- Social Graph
- Collaborative Playlists
- Activity Feed

Auth y Usuarios

- OAuth 2.0 + SSO
- Planes: Free, Premium, Duo, Family
- Payment Service multi-país

Plataforma de Datos

- ~1 billón eventos/día
- Scio (Scala + Apache Beam)
- Feature Store centralizado
- ABBA: experimentación A/B masiva

■ Sección 1: Arquitectura de Spotify

▤ Content Platform

Ingestión de Contenido

- Ingestion Pipeline automatizado
- Recepción de sellos discográficos
- Distribuidores digitales integrados

Herramientas para Creadores

- Spotify for Artists
- Spotify for Labels
- Spotify for Podcasters
- Rights Management (regalías, licencias)

■ Sección 1: Arquitectura de Spotify

▒ Evolución Histórica

- **Fase 1 – Monolito (2006-2012):** Python/Django, C++, PostgreSQL
- **Fase 2 – SOA incipiente (2012-2015):** Cassandra, Kafka, Docker, squads
- **Fase 3 – Cloud + microservicios (2016-2018):** GCP, Kubernetes, ~1.500 servicios
- **Fase 4 – Madurez (2018-2020):** GraphQL, gRPC, Backstage open source
- **Fase 5 – Expansión e IA (2021-presente):** Podcast, DJ AI, >1B eventos/día

■ Sección 1: Arquitectura de Spotify

▤ Manejo de Datos

- **PostgreSQL**: perfiles, suscripciones, facturación
- **Apache Cassandra**: historial de reproducción, eventos masivos
- **Apache Kafka**: cada interacción genera eventos → consumidores
- Procesamiento en **tiempo real** sin afectar experiencia del usuario
- Desacoplamiento total entre microservicios

■ Sección 1: Arquitectura de Spotify

■ Seguridad

- **OAuth 2.0**: autenticación de usuarios y apps de terceros
- **Mínimo privilegio**: cada microservicio accede solo a lo necesario
- **TLS + HTTPS**: cifrado en tránsito
- **AES-256**: cifrado en reposo
- **DRM**: protección de contenido con licencias y restricciones geográficas

■ Sección 1: Arquitectura de Spotify

▒ Escalabilidad y Disponibilidad

Escalabilidad

- Kubernetes con autoescalado
- CDN multi-proveedor global
- Sharding de bases de datos
- Escalado independiente por servicio

Disponibilidad

- Replicación multi-datacenter
- Self-healing con Kubernetes
- Prometheus + Grafana
- Backups periódicos
- Recuperación ante desastres



Sección 1: Arquitectura de Spotify

Diagrama de Arquitectura (2)



■ Sección 1: Arquitectura de Spotify

▤ Decisiones Arquitectónicas Clave

▥ ADR-1: Monolito → Microservicios

- **Contexto:** monolito Python/Django era cuello de botella organizacional
- **Decisión:** descomponer en microservicios independientes
- **Resultado:** 250+ squads despliegan en paralelo, time-to-market reducido
- **Contrapartida:** complejidad en orquestación, service discovery, consistencia eventual

■ Sección 1: Arquitectura de Spotify

▒ Decisiones Arquitectónicas Clave

▒ ADR-2: Google Cloud | ADR-3: Cassandra

ADR-2: Google Cloud

- 5.000 servidores → GCP
- Reducción de TCO
- Acceso a servicios managed
- Riesgo: vendor lock-in

ADR-3: Cassandra

- PostgreSQL no escalaba horizontalmente
- Arquitectura masterless
- Replicación multirregión
- Contrapartida: consistencia eventual

■ Sección 1: Arquitectura de Spotify

■ Decisiones Arquitectónicas Clave

■ ADR-4: Apache Kafka | ADR-5: Apollo GraphQL

ADR-4: Apache Kafka

- Desacopla productores y consumidores
- Garantiza orden y durabilidad
- Permite reprocesamiento histórico
- Contrapartida: complejidad operativa

ADR-5: Apollo GraphQL

- Cada cliente declara sus datos
- Optimiza ancho de banda
- Reduce latencia percibida
- Contrapartida: DataLoaders necesarios

■ Sección 1: Arquitectura de Spotify

■ Decisiones Arquitectónicas Clave

■ ADR-6: Modelo de Squads | ADR-7: Backstage

ADR-6: Modelo de Squads

- 5-8 ingenieros por squad
- "You build it, you run it"
- Ley de Conway aplicada
- Riesgo: fragmentación tecnológica

ADR-7: Backstage

- Catálogo centralizado de servicios
- Documentación viva desde código
- Scaffolding automatizado
- Donado a CNCF en 2020

■ Sección 1: Arquitectura de Spotify

▒ Decisiones Arquitectónicas Clave

▒ ADR-8: Caché Distribuida (Redis + Memcached)

- **Contexto:** streaming exige latencias < 200 ms; consultar BD por reproducción es inviable
- **Decisión:** capa de caché distribuida con Redis y Memcached para sesión y catálogo frecuente
- **Resultado:** reducción drástica de carga sobre Cassandra y PostgreSQL
- **Contrapartida:** inconsistencia temporal caché-BD, gestionada con TTL e invalidación diferenciada

■ Sección 1: Arquitectura de Spotify

▤ Conclusión

- Arquitectura de **microservicios event-driven** diseñada para escala global
- **Tecnologías:** Kubernetes, Kafka, Cassandra, PostgreSQL, GCP
- **Decisiones clave** con ADRs documentados y contrapartidas gestionadas
- **Alineación** entre estructura organizacional (squads) y arquitectura técnica
- Sistema **confiable, flexible y preparado** para evolucionar

■ Sección 2: Rol del Arquitecto de Software Actual

▒ Perfiles Analizados

- **Spotify** – Staff Software Architect (Suecia, Híbrido)
- **Globant** – Software Architect Cloud Solutions (EE.UU., Remoto)
- **Mercado Libre** – Arquitecto de Software Senior (Argentina, Híbrido)

Fuentes: LinkedIn, Glassdoor, Get on Board (2026)

■ Sección 2: Rol del Arquitecto de Software Actual

▒ Coincidencias Clave

- **Cloud e Infraestructura Inmutable:** dominio obligatorio de AWS/GCP/Azure, Docker y Kubernetes
- **Sistemas de Datos:** Apache Kafka, bases distribuidas, Redis
- **Habilidades Blandas:** mentoría, negociación, liderazgo de influencia
- El arquitecto moderno pasa gran parte de su tiempo **comunicando e influyendo**

■ Sección 2: Rol del Arquitecto de Software Actual

▒ Diferencias Significativas

- **Escala organizacional:** Spotify → hiperescala; Globant → consultoría y FinOps
- **Brecha salarial:** Norteamérica/Europa (\$120K-\$185K) vs LATAM (\$60K-\$75K)
- Mismas exigencias técnicas conceptuales, **compensaciones ligadas a la ubicación**

■ Sección 2: Rol del Arquitecto de Software Actual

■ Conclusión

- El arquitecto evolucionó de **diseñador técnico** a **posición estratégica**
- Conocimientos requeridos: microservicios, cloud, DevOps, ciberseguridad
- Habilidades blandas **tan importantes** como las técnicas
- Rol de **punto entre negocio y tecnología**

■ Sección 3: El Rol del Arquitecto en el Futuro

■ Transformación del Rol

El arquitecto pasará de **diseñador de estructuras técnicas** a **estratega tecnológico** que toma decisiones sobre sistemas:

- Más inteligentes y automatizados
- Más distribuidos
- Más regulados
- Con impacto ético, económico, ambiental y social

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Impacto de la IA

Oportunidades

- Análisis masivo de información técnica
- Comparación de patrones arquitectónicos
- Detección de deuda técnica
- Documentación automática
- Optimización de costos cloud

Desafíos

- Precisión y alucinaciones
- Privacidad de datos
- Falta de marcos de evaluación
- La IA apoya, no decide

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Impacto de la Computación Cuántica

- IBM proyecta computadora cuántica tolerante a fallos hacia **2029**
- Oportunidades: optimización de distribución, recomendaciones, análisis de datos
- Adopción **gradual y limitada** a casos concretos
- **Impacto inmediato en seguridad:** migración a criptografía post-cuántica
- NIST publicó estándares de criptografía post-cuántica en 2024

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Nuevas Responsabilidades

- **Gobernanza de IA:** sistemas auditables, explicables y controlados (ISO/IEC 42001:2023)
- **Sostenibilidad:** eficiencia energética, reducción de huella de carbono (SCI Specification)
- **Ética tecnológica:** privacidad, transparencia, responsabilidad
- **Cumplimiento normativo:** regulaciones emergentes sobre IA y datos
- **Gestión de riesgos:** anticipar consecuencias no evidentes

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Habilidades a Desarrollar

Técnicas

- Arquitectura cloud nativa
- MLOps y pipelines de ML
- Gobernanza de datos
- IA generativa
- Criptografía post-cuántica
- Observabilidad avanzada

Humanas

- Pensamiento crítico
- Liderazgo y negociación
- Visión de negocio
- Comunicación estratégica
- Análisis de trade-offs

■ Sección 3: El Rol del Arquitecto en el Futuro

■ Automatización vs. Criterio Humano

Automatizable con IA

- Documentación técnica
- Diagramas iniciales
- Análisis de dependencias
- Detección de deuda técnica
- Pruebas y reportes

Requiere criterio humano

- Prioridades estratégicas
- Resolución de conflictos
- Interpretación del negocio
- Evaluación ética
- Decisiones de seguridad
- Migraciones complejas

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Valor Diferencial del Arquitecto

- **Conectar** tecnología, negocio y responsabilidad social
- **Formular las preguntas correctas**, no solo generar opciones
- **Equilibrar**: innovación ↔ estabilidad, automatización ↔ control, personalización ↔ privacidad
- **Anticipar impactos** y diseñar arquitecturas adaptables
- No será solo un experto técnico: será un **líder de decisiones complejas**

■ Sección 3: El Rol del Arquitecto en el Futuro

▒ Riesgos y Oportunidades

Oportunidades

- Herramientas más avanzadas
- Automatización de tareas repetitivas
- Más tiempo para decisiones estratégicas
- Nuevos enfoques (cuántica)

Riesgos

- Dependencia excesiva de la IA
- Deuda técnica acelerada
- Sesgos algorítmicos
- Aumento del consumo energético
- Dificultad regulatoria

■ Sección 3: El Rol del Arquitecto en el Futuro

■ Conclusión

- El arquitecto **no desaparecerá**: se volverá más estratégico, interdisciplinario y crítico
- La IA automatiza tareas, pero el **criterio humano** sigue siendo indispensable
- Deberá diseñar sistemas que integren IA, prepararse para seguridad post-cuántica y garantizar confianza
- **La diferencia**: saber cuándo usar nuevas tecnologías, cómo gobernarlas y qué consecuencias generan

■ Conclusión General

■ Síntesis del Trabajo

- Spotify demuestra que una **arquitectura bien diseñada** habilita escala global e innovación continua
- El arquitecto moderno es un **punto estratégico** entre negocio y tecnología
- El futuro exige **nuevas competencias**: IA, ciberseguridad, sostenibilidad, ética
- El arquitecto seguirá siendo **indispensable** como líder de decisiones complejas

■ Referencias

- Kniberg, H. y Ivarsson, A. (2012). Scaling Agile @ Spotify
- Kreitz, G. y Niemelä, F. (2010). Spotify – Large scale, low latency, P2P streaming
- Newman, S. (2021). Building Microservices (2.^a ed.). O'Reilly
- Esposito et al. (2025). Generative AI for Software Architecture. arXiv
- ISO/IEC 42001:2023. AI Management System
- NIST (2024). Post-Quantum Cryptography Standards
- IBM (2025). Fault-Tolerant Quantum Computing Roadmap
- Spotify Engineering (2024). Engineering Blog
- Apache Kafka, Cassandra, Kubernetes Documentation (2024)
- Google Cloud Architecture Framework (2024)

■ ¡Gracias!

¿Preguntas?